AI COMPONENT · CODE & CONCEPTS (BEGINNER LEVEL)

Error Classification with NLP

How the platform labels the TYPE of a mistake — explained from your code

Intelligent Adaptive Learning Platform · Yassine Ben Nessib · EspritTech — explained from the real source code

> **In one sentence —** A trained classifier reads a short text describing a mistake and predicts **which of 11 error types** it is, with a confidence. If unsure, it says "unknown" instead of guessing. Code: `error_classification_service.py` (use) + `train_error_classifier.py` (training).

1 The idea in one minute

Knowing an answer is wrong is not enough — we want to know **why**: a sign error, an arithmetic slip, a wrong method... Each needs a different hint. So we train a model to put a mistake into one of **11 categories** (the taxonomy in `error_taxonomy.py`).

2 Where it lives in the code

`app/services/error_classification_service.py`
Uses the trained model to label a mistake.

`scripts/train_error_classifier.py`
Trains the model and saves it.

`app/data/error_classifier.joblib`
The saved trained model (the "brain").

`app/data/error_taxonomy.py`
The 11 error types (the labels).

3 Training: turning text into features

A model cannot read words, only numbers. So we first turn each text into a **feature vector**. Your code tries the smart way first (embeddings), and falls back to the classic way (TF-IDF).

TRAIN_ERROR_CLASSIFIER.PY — BUILD_FEATURES()

```
def build_features(texts):
    try:
        from app.services import embedding_service
        if embedding_service.is_available():
            X = np.vstack([embedding_service.encode(t) for t in texts])
            return X, "embeddings", None        # PRIMARY: semantic vectors
    except Exception:
        pass
    vec = TfidfVectorizer(ngram_range=(1, 2), min_df=2, lowercase=True)
    X = vec.fit_transform(texts)                 # FALLBACK: TF-IDF
    return X, "tfidf", vec
```

What this does, step by step:

- **Primary path:** if embeddings are available (Brick 2), each text becomes a meaning-vector. `np.vstack` stacks them into a matrix `X` (one row per example).
- **Fallback:** otherwise, `TfidfVectorizer` turns text into numbers using **TF-IDF** — it weights each word/2-word group by how informative it is (frequent here but rare overall = important).
- It returns `X` (the numbers), the `method` used, and the vectorizer (saved so we transform new text the same way).

4 Training the classifier

TRAIN_ERROR_CLASSIFIER.PY — FIT & SAVE

```
X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y)
clf = LogisticRegression(max_iter=2000, C=4.0)
clf.fit(X_train, y_train)                  # learn from the training part
acc = accuracy_score(y_test, clf.predict(X_test))   # check on unseen part
...
bundle = {"method": method, "vectorizer": vec, "clf": clf_full,
          "classes": list(clf_full.classes_), "threshold": 0.30}
joblib.dump(bundle, OUT)                    # save the trained model
```

What this does, step by step:

- `train_test_split` splits the data: 80% to **learn**, 20% kept aside to **test** honestly. `stratify=y` keeps the class balance in both parts.
- `LogisticRegression` is a simple, fast, explainable classifier — the standard baseline for text. `clf.fit(...)` is where it actually **learns**.
- `accuracy_score` measures how often it is right on the unseen test set (plus a per-class report and 5-fold cross-validation).
- Finally everything (method, vectorizer, classifier, labels, threshold) is packed into a `bundle` and saved with `joblib.dump` to the `.joblib` file.

5 Using the model at runtime

ERROR_CLASSIFICATION_SERVICE.PY — CLASSIFY()

```
def classify(text, lang="en"):
    bundle = _load()
    if bundle["method"] == "embeddings":
        vec = embedding_service.encode(text)
        X = np.asarray(vec).reshape(1, -1)      # same features as training
    else:
        X = bundle["vectorizer"].transform([text])
    proba = bundle["clf"].predict_proba(X)[0]   # probability for each class
    idx = int(np.argmax(proba))                 # most likely class
    confidence = float(proba[idx])
    code = bundle["classes"][idx]
    if confidence < bundle["threshold"]:        # not sure enough?
        return {"code": "unknown", "confidence": confidence}
    return {"code": code, "confidence": confidence}
```

What this does, step by step:

- It vectorizes the new text **exactly like training** (embeddings or TF-IDF) — this is crucial, the model only understands the same kind of numbers.
- `predict_proba` gives a probability for **each** of the 11 error types.
- `np.argmax` picks the index of the highest probability; `classes[idx]` turns it back into a readable label.
- **Safety net:** if the top probability is below the threshold (0.30), it returns `"unknown"` instead of guessing wrongly.

TF-IDF WEIGHT OF A WORD (WHY A WORD IS “IMPORTANT”)

$$\text{tfidf}(w, d) = \text{tf}(w, d) \times \log\left(\frac{N}{\text{df}(w)}\right)$$

tf = times w appears in this text · N = number of texts · df = texts containing w

6 Worked example

From a wrong answer to a precise hint

A student gets the right value but the **opposite sign**. `classify()` returns `{"code":"sign_error", "confidence":0.82}`. The platform shows a targeted hint: “check the sign when you apply the bounds”, instead of a vague “incorrect”.

7 Strengths & limits

Strengths

- Turns “wrong” into a precise diagnosis
- Fast and cheap
- Explainable
- “unknown” safety net avoids bad guesses

Limits

- Needs labelled examples
- Only knows trained error types
- Quality depends on the labels
- New mistakes need retraining

Honest note — The model was trained on a **synthetic** labelled set (examples generated per error type by `generate_error_data.py`). The honest next step is to label and train on **real** student answers and report the test accuracy.

8 Q&A

DEFENSE / INTERVIEW QUESTIONS (INCLUDING “WHAT DOES THIS BLOCK DO?”)

Q What does classify() do, line by line?

A It loads the model, vectorizes the text the same way as training, gets a probability per class, takes the most likely one, and — if confident enough — returns that error type; otherwise “unknown”.

Q What is predict_proba vs predict?

A predict gives only the chosen class; predict_proba gives a probability for every class, which lets us measure confidence and apply a threshold.

Q Why a threshold and an “unknown” class?

A To avoid confidently wrong labels. If no class is clearly likely, saying “unknown” and falling back is safer than a wrong hint.

Q What is TF-IDF and why use it?

A A way to turn text into numbers by weighting informative words. It is the classic, lightweight fallback when embeddings are not available.

Q Why Logistic Regression?

A Simple, fast, interpretable, and strong on small text datasets with good features — the standard baseline, easy to defend.

Q How do you evaluate it?

A train_test_split keeps 20% unseen; we report accuracy, a per-class report, and 5-fold cross-validation.

9 Key terms

Feature vector — The numbers that represent a text for the model.

TF-IDF — Word-weighting that turns text into numbers.

train_test_split — Split data into a learning part and an honest test part.

predict_proba — Probability the model gives to each class.

argmax — The index of the largest value.

Threshold — Minimum confidence to accept a prediction.

joblib — Library to save/load a trained model to a file.

Logistic Regression — A simple, explainable classifier.